

数字电路设计基础

浙江大学电工电子教学中心 张德华

■ 实验目的

- 了解数字电路基础知识
- 掌握简单数字逻辑电路的设计
- 学习FPGA开发工具Quartus Prime Lite的使用

□ 基本逻辑关系

■ 逻辑变量和逻辑变量的命名

- 只有二个状态的变量称为逻辑变量

真/假, 开/关, 启动/停止, 0/1

- 命名: A, B, C, Z, STOP等

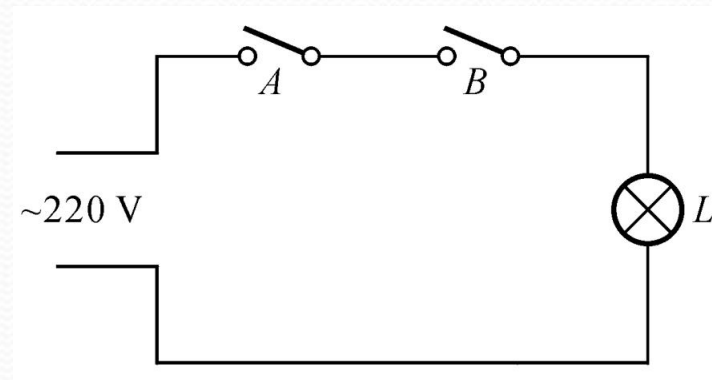
■ 基本逻辑关系

- 逻辑与, 逻辑或, 逻辑非

■ 逻辑与运算

- 逻辑变量A和B “与” 等于L
- 逻辑 “与” 的真值表描述
- 物理世界 “与” 运算

A	B	L
0	0	0
0	1	0
1	0	0
1	1	1



■ 逻辑与运算

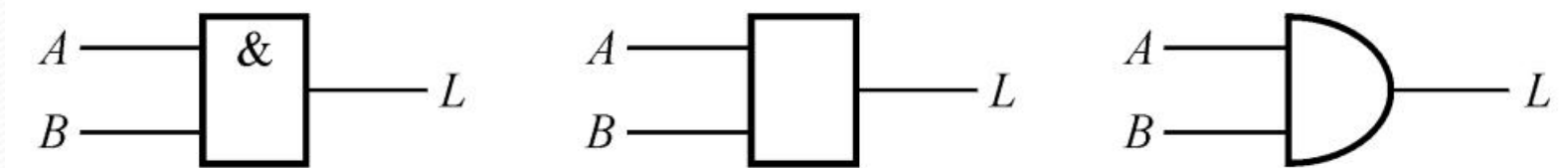
- “与”运算规则： $0 \cdot 0 = 0$; $0 \cdot 1 = 0$;
 $1 \cdot 0 = 0$; $1 \cdot 1 = 1$;

A	B	L
0	0	0
0	1	0
1	0	0
1	1	1

- 逻辑“与”的数学表达式

$$L = A \cdot B \text{ 或 } A \& B \text{ 或 } A \text{ and } B$$

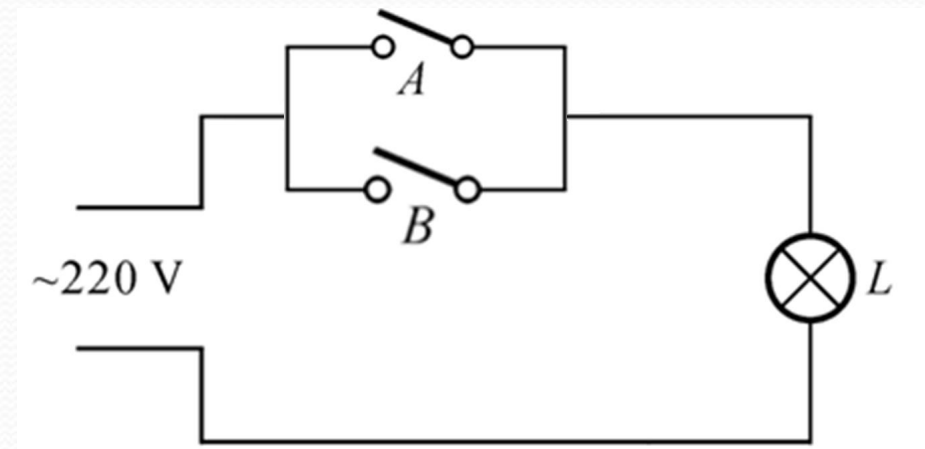
- 逻辑“与”的电路实现---与门----电路符号



■ 逻辑或运算

- 逻辑变量A和B “或” 等于L
- 逻辑 “或” 的真值表描述
- 物理世界 “或” 运算

A	B	L
0	0	0
0	1	1
1	0	1
1	1	1



■ 逻辑或运算

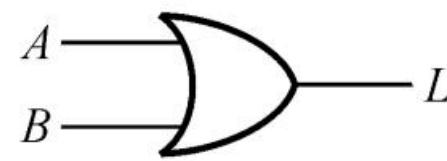
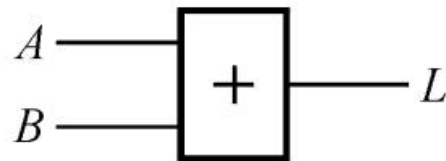
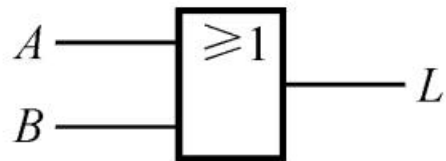
- “或”运算规则： $0+0=0$ ； $0+1=1$ ；
 $1+0=1$ ； $1+1=1$ ；

- 逻辑“或”函数表达：

$$L=A+B \text{ 或 } A \# B \text{ 或 } A \text{ or } B$$

A	B	L
0	0	0
0	1	1
1	0	1
1	1	1

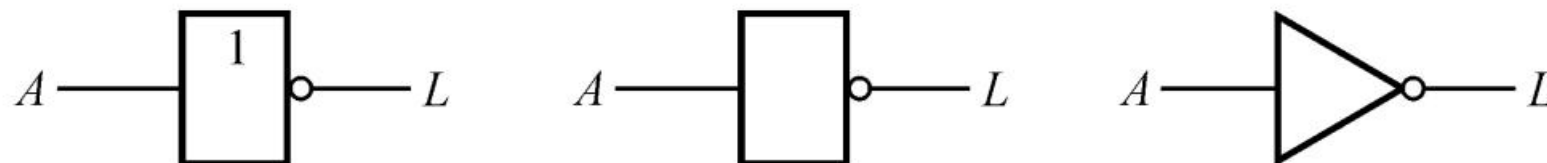
- 逻辑“或”的电路实现---或门----电路符号



■ 逻辑非运算

- 逻辑变量A的“非”等于L
- 逻辑“非”的真值表描述
- “非”运算规则： $\bar{0} = 1; \bar{1} = 0$
- 逻辑“非”函数表达： $L = \bar{A} = !A = not\ A$
- 逻辑“非”的电路实现---非门----电路符号

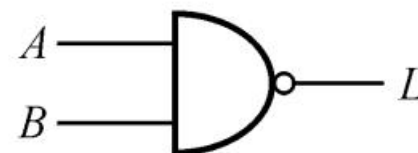
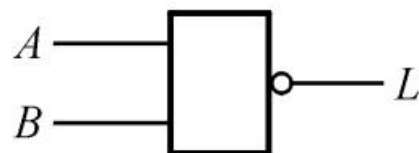
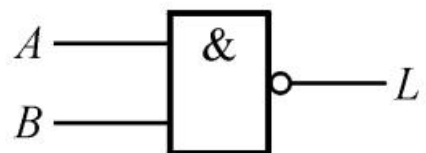
A	L
0	1
1	0



■ 复合逻辑运算

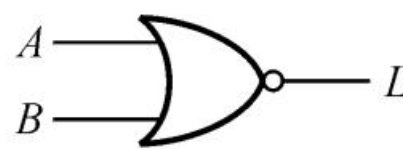
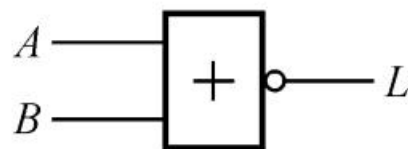
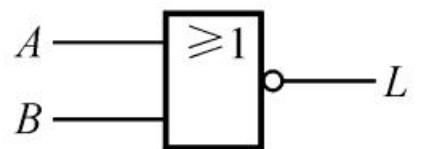
• “与非” 运算

$$L = \overline{A \cdot B}$$



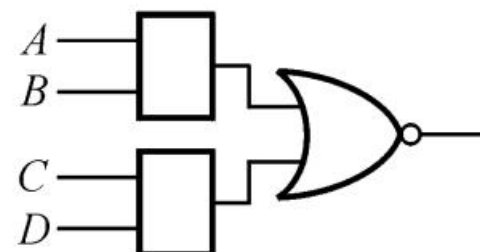
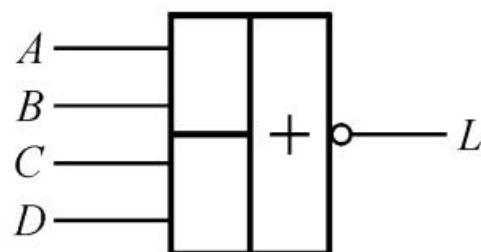
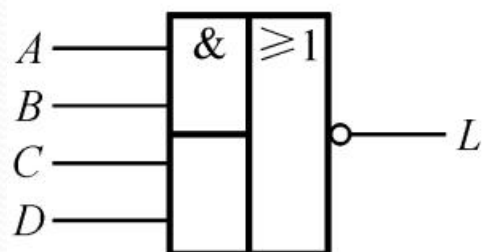
• “或非” 运算

$$L = \overline{A + B}$$



• “与或非” 运算

$$L = \overline{A \cdot B + C \cdot D}$$

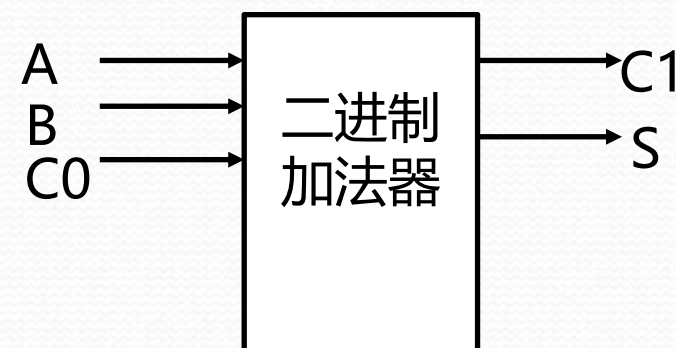


■ 复杂逻辑关系的真值表描述

【例】设计一逻辑电路实现一位二进制数加法运算

- 输入：被加数A，加数B，输入进位C0
- 输出：和S，进位C1
- 根据加法原理列真值表

A	B	C ₀	C ₁	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



■ 真值表描述转换成逻辑函数

【例】设计一逻辑电路，当输入A与B不同时输出Z为逻辑1，相同时输出逻辑0（异或运算）。

- 根据题意得真值表

A	B	Z
0	0	0
0	1	1
1	0	1
1	1	0

- 当A=0同时（与）B=1时，Z等于1，即 $Z = \bar{A} \cdot B \quad \{\bar{0} \cdot 1 = 1\}$
- 或当A=1同时（与）B=0时，Z等于1，即 $Z = A \cdot \bar{B} \quad \{1 \cdot \bar{0} = 1\}$
- 真值表转换成逻辑函数的结果

$$Z = \bar{A} \cdot B + A \cdot \bar{B}$$

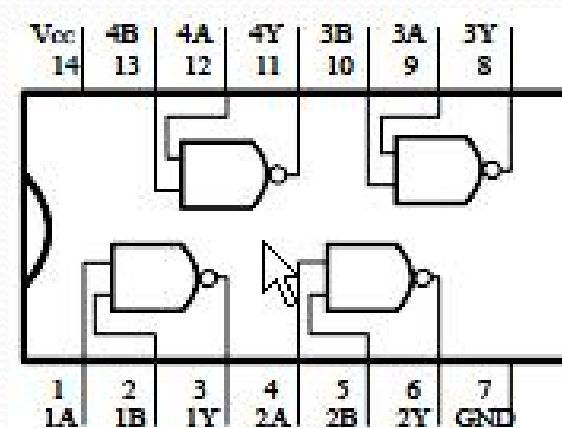
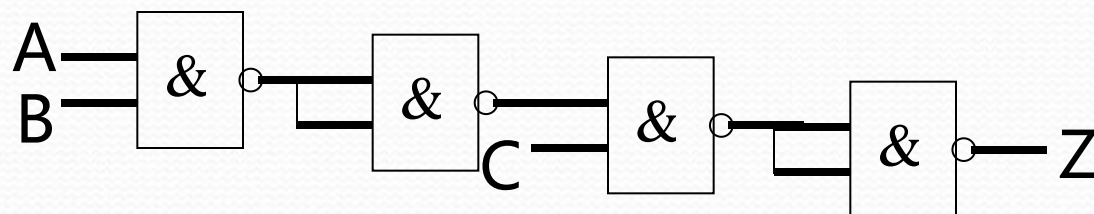
■ 用门电路实现逻辑函数

【例】用74LS00二输入端与非门实现 $Z = A \cdot B \cdot C$

- 将Z转换成与非—与非形式

$$Z = A \cdot B \cdot C = \overline{\overline{A \cdot B \cdot C}}$$

$$= \overline{\overline{A} \cdot \overline{B} \cdot \overline{C}}$$



- 如果要求用二输入端的或非门实现怎么办？
- 结论：用门电路综合或实现逻辑函数的过程十分麻烦？

□ 可编程逻辑器件(PLD)

1、集成逻辑器件的分类



2. 集成逻辑器件的特点

➤ 标准逻辑器件

使用方便、价格便宜，但其规模一般较小，一个数字系统往往要用几十片甚至上百片标准逻辑器件来完成；

➤ 含CPU的微处理器

在时钟脉冲作用下不断执行用户的软件程序，用户编程并不对其硬件结构产生影响，它的工作速度一般较低；

➤ 半定制、全定制ASIC

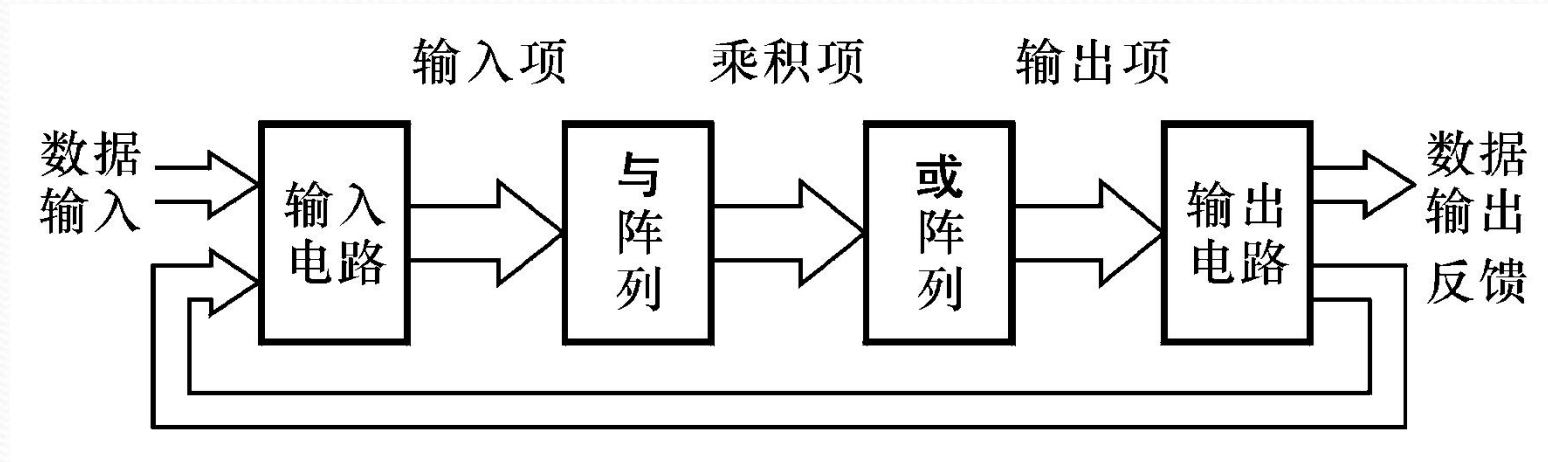
产品开发需要半导体厂家参与，周期长、费用高，其开发不可能普及；

➤ 可编程逻辑器件

(1) 逻辑功能可编程。(2) 器件规模很大。(3) 工作速度很高。(4) 使用相对复杂。

□ 可编程逻辑器件(PLD)

3. PLD的基本结构



- 输入电路产生输入变量的原变量和反变量
- 与阵列产生输入变量的与项（乘积项）
- 或阵列对乘积项有选择地进行或运算
- 输出电路产生输出信号，提供反馈信号

□ 可编程逻辑器件(PLD)

- ◆ PLD器件内部有大量可编程 “与阵列” 和 “或阵列”
- ◆ 与或阵列规模越大，实现Z时越简单
- ◆ PLD器件提供的EDA工具能自动完成逻辑实现
- ◆ 用户的主要工作是完成逻辑功能描述
- ◆ 逻辑功能描述方法
 - 原理图描述法
 - HDL语言描述法
 - 高级C语言描述法

■ 简明VHDL语言

◆ 最简VHDL语言构成

- 库引用
- 实体说明 (I/O定义)
- 结构体说明 (逻辑功能描述)

◆ 【例】

右图VHDL源文件

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
```

库引用

```
entity test is
    Port ( A : in  STD_LOGIC;
          B : in  STD_LOGIC;
          Z : out STD_LOGIC);
end test;
```

I/O定义

```
architecture Behavioral of test is
begin
    Z<=A and B;
end Behavioral;
```

功能描述

■ 库引用部分

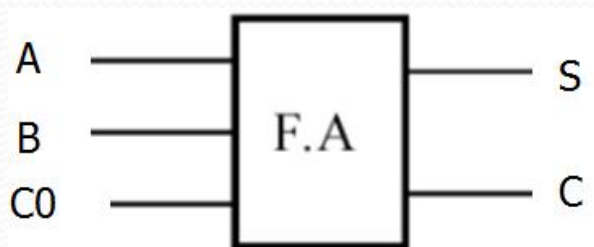
- 相当于C语言中头文件，用于关键词常用规范定义
- 最典型的是IEEE标准库
- 一般VHDL源文件的库说明和库引用

```
library IEEE;  
use IEEE.std_logic_1164.all;  
use IEEE.std_logic_unsigned.all;  
USE IEEE.numeric_std.ALL;  
USE IEEE.std_logic_arith.all;
```

■ 实体说明部分

- 实体说明用于定义逻辑设计的I/O
- I/O信号也称端口信号

【例】一位二进制全加器



```
entity add1bit is  
    port (  A, B, C0: in STD_LOGIC;  
            S, C1: out STD_LOGIC;  
end add1bit ;
```


■ 结构体部分

- ◆ 结构体部分描述逻辑功能
- ◆ 结构体部分组成
 - 说明语句
 - 各种并发语句

ARCHITECTURE 构造体名 OF 实体名 IS

[说明部分]

BEGIN

并发处理语句1

并发处理语句2

... ..

END构造体名

并发是VHDL语言区别C语言的重要特征!

■ 结构体中的说明语句

- 定义设计过程需要的中间信号
- 声明需要用到的外部元件 (**另一个独立的VHDL源码**)

```
component qcjFDC
  Port ( D : in  STD_LOGIC;
        CLK : in  STD_LOGIC;
        CLRn : in  STD_LOGIC;
        Q : out  STD_LOGIC);
end component;
```

元件引用

```
signal D0,D1,RDn0:STD_LOGIC:='0';
signal Q0:STD_LOGIC:='0';
signal Q1:STD LOGIC:='1';
```

中间信号定义

■ 最常用的三个并发语句

- 代入语句
- 元件调用语句
- 进程语句

◆ 代入并发语句

- 适合将逻辑函数表达式Z转换成VHDL语句

【例】 $Z = \bar{A} \cdot B + \bar{B} \cdot C$

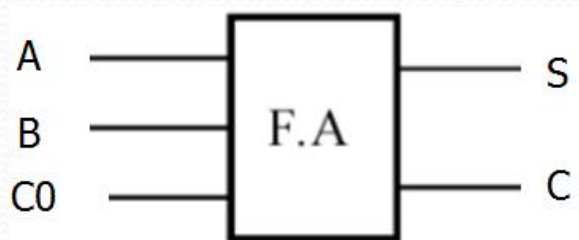
对应VHDL语言描述：

$Z <= (\text{ not } A \text{ and } B) \text{ or } (\text{ not } B \text{ and } C)$

◆ 元件调用并发语句

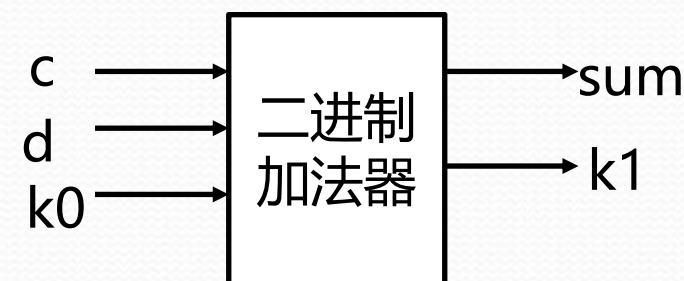
- 元件调用语句相当于C语言中的函数调用

【例】若在结构体说明部分声明过一位二进制加法器



```
component add1bit is
  port (  A, B, C0: in STD_LOGIC;
          S, C1: out STD_LOGIC;
end component ;
```

- 并发元件调用过程



法1: U1: add1bit *port map* (c, d, k0, sum, k1)

法2: U1: add1bit *port map* (A=>c, B=>d, C0=>k0, S=>sum, C1=>k1)

法3: U1: add1bit *port map* (C0=>k0, A=>c, B=>d, S=>sum, C1=>k1)

◆ 进程并发语句

• 进程结构

```
[进程标号]: PROCESS ([敏感信号列表])  
    [说明语句];  
    BEGIN  
        顺序语句1;  
        顺序语句2; ...  
    END PROCESS [进程标号];
```

- **当敏感信号发生变化时执行进程**
- 进程说明语句：最常见是定义进程中需用到的中间信号
- 进程内部有并发代入语句
- 进程内部有顺序语句：如if then else;case等

■ 进程语句示例1

```
p1:process (A,B)
BEGIN
  IF (A='1')
    THEN Z<=not B;
  ELSE
    Z<=B;
  END IF;
END PROCESS;
```

- 上述描述等效实现 $Z = A \cdot \overline{B} + \overline{A} \cdot B$
- 若采用代入并行语句实现结果是：

```
Z<=(A and not B) or (not A and B);
```


■ 进程语句示例2

```
PROCESS (A,B)
  variable tmp:STD_LOGIC_vector(1 downto 0);
BEGIN
  tmp:=(A&B);--A与B并置成二位二进制
  CASE (tmp) IS
    WHEN "00"=>Z<='0';
    WHEN "01"=>Z<='1';
    WHEN "10"=>Z<='1';
    WHEN "11"=>Z<='0';
    WHEN others=>Z<='0';
  END CASE;
END PROCESS;
```

A	B	Z
0	0	0
0	1	1
1	0	1
1	1	0

- 上述描述对应真值表
- 利用进程中的CASE语句，适合将真值表描述直接转换成VHDL语句

VHDL源码示例

设计一个计数器/分频器

--对输入时钟F100MHZ进行DIV_NUM倍分频

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
use IEEE.STD_LOGIC_ARITH.ALL;
```

```
use IEEE.STD_LOGIC_UNSIGNED.ALL;
```

```
entity VAR_CLK is
```

```
    Port ( F100MHZ : in  STD_LOGIC;
```

```
          DIV_NUM : natural range 0 to 100000000; --分
```

```
          freq_out : out  STD_LOGIC);
```

```
end VAR_CLK;
```

```
architecture Behavioral of VAR_CLK is
```

```
begin
```

```
    P1:process (F100MHZ)
```

```
        variable cnt: natural range 0 to 100000000;    --分频计数变量
```

```
    begin
```

```
        if rising_edge(F100MHZ) then
```

```
            if cnt=DIV_NUM then
```

--当cnt计到DIV_NUM后返回0

```
                cnt:=0;
```

```
            elsif cnt<DIV_NUM/2 then
```

```
                freq_out<='0';
```

```
                cnt:=cnt+1;    --计数加1
```

```
            else
```

```
                freq_out<='1';
```

```
                cnt:=cnt+1;    --计数加1
```

```
            end if;
```

```
        end if;
```

```
    end process P1;
```


Intel公司的FPGA开发工具

■ Quartus Prime Lite 集成开发环境

■ 设计流程

- 前端设计(器件无关)

✓ 编写逻辑功能的VHDL源码

*编写仿真测试VHDL源码

✓ 逻辑简化和综合

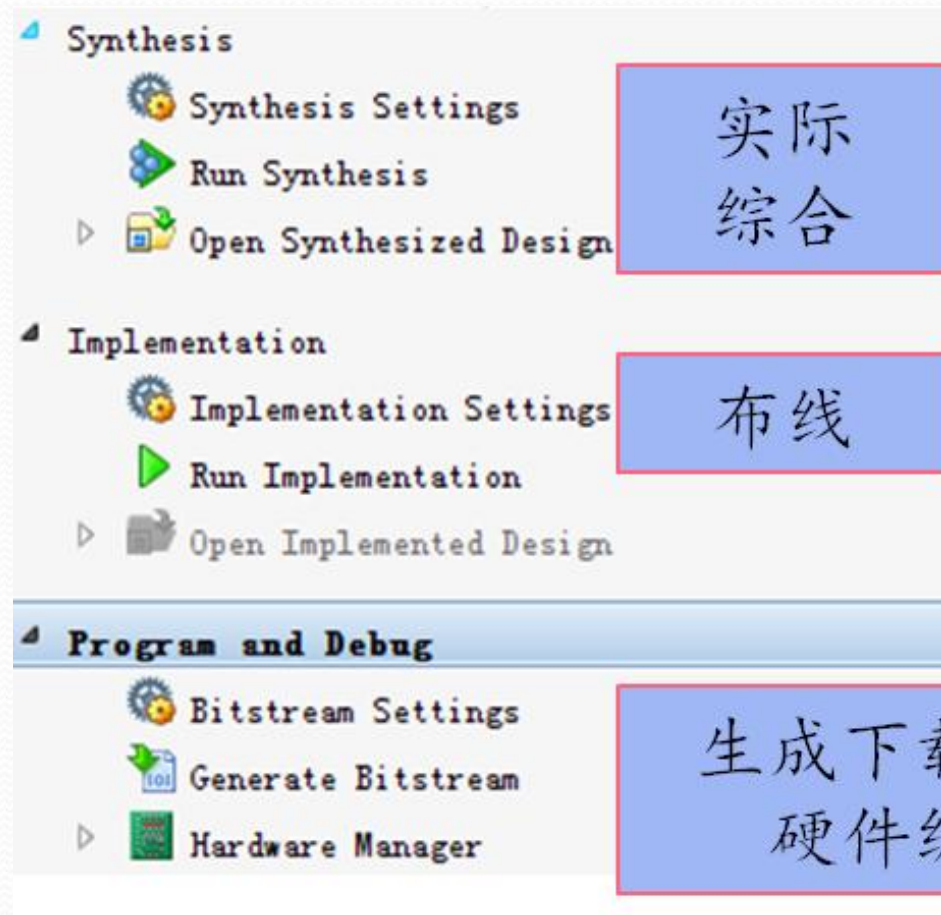


- 后端设计(器件相关)

- ✓ 锁定输入、输出引脚

- ✓ 逻辑综合和布线

- ✓ 下载设计结果



□ 实验内容

用Quartus Prime Lite 软件和DE10-Lite开发板设计并实现如下逻辑功能

- (1) 输入是三位二进制数A,B,C, 要求当输入是2或3的倍数时输出等于逻辑1, 其它情况, 输出等于0
- (2) 设计并实现一位二进制全加器

□ 探究题

某客厅四周有 4 个房间，每个房间门口有一个开关，客厅中间有一盏灯 A

试设计一个逻辑电路，要求每个开关都能控制灯 A 的亮灭，并在 DE10 开发板上模拟此逻辑电路的功能。